NETWORKING

REST API Design Visual Guide

Resources, verbs, and status codes — APIs that feel inevitable.

ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Resources & URIs

Nouns, not verbs — the URL names the THING.

```
GET  /orders/42            ← a resource
GET  /orders/42/items      ← a sub-resource
GET  /getOrder?id=42       ← RPC smell: avoid
```

- Plural nouns, kebab-case: /users, /order-items — consistent everywhere.
- Hierarchy for ownership: /users/7/orders — but don't nest deeper than 2 levels.
- Filtering/sorting via query params: /orders?status=paid&sort=-created.

STEP 2/6

HTTP Verbs: Meaningful Methods

The verb says what; the URL says what to.

GET	read — safe, idempotent, cacheable
POST	create (or action) — neither safe nor idempotent
PUT	replace whole resource — idempotent
PATCH	partial update — idempotent if designed so
DELETE	remove — idempotent (deleting twice = deleted)

Idempotency keys on POST: clients retry safely — the server dedupes by key. Essential for payments.

STEP 3/6

Status Codes: Speak HTTP

The code is the first thing clients read — choose precisely.

2xx — success

200 OK, 201 Created (+ Location), 204 No Content

4xx — client error

400 bad input, 401 unauthenticated, 403 forbidden, 404 not found, 409 conflict, 429 slow down

5xx — server error

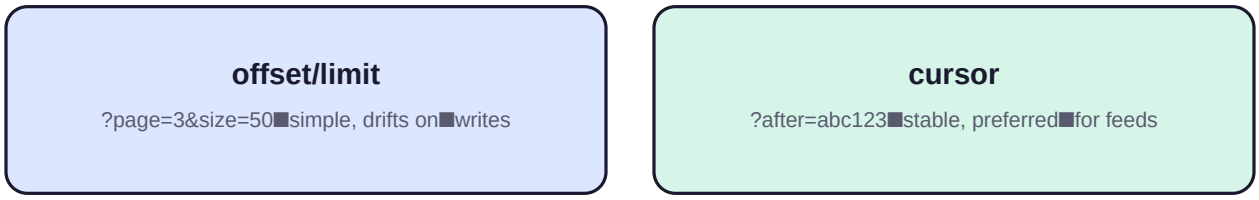
500 bug, 502 bad gateway, 503 overloaded (+ Retry-After)

Never 200-with-error-body: intermediaries, monitors, and client libraries all key off the status code.

STEP 4/6

Pagination: Never Dump the Table

Unbounded lists are a DoS you wrote yourself.



- Always paginate list endpoints — default page size, max cap (e.g. 100).
- Cursor pagination survives inserts/deletes; offset skips or duplicates rows on live data.
- Return metadata: total count (when cheap), next cursor, and Link headers for discoverability.

STEP 5/6

Versioning Strategies

APIs are promises — evolve without breaking them.

URL versioning

/v1/orders — explicit, obvious, most common

Header versioning

Accept: application/vnd.api.v2+json — clean URLs, harder to debug

Evolve, don't version

additive changes only; deprecate with Sunset headers

Rule: never remove or retype a field in a shipped version — additive only, deprecate loudly.

STEP 6/6

HATEOAS: Hypermedia Controls

Responses that teach clients what to do next.

```
{ "id": 42, "status": "paid",
  "_links": { "self": "/orders/42",
              "refund": "/orders/42/refund" } }
```

- Links encode valid state transitions — clients discover actions instead of hardcoding URLs.
- Full HATEOAS is rare in practice; link relations on key resources capture most of the value.
- Pairs well with versioning-by-evolution: clients follow links, not versioned URL templates.

STEP ✓/✓

Key Takeaways

REST API Design — Remember

- Resources are nouns in the URL; verbs carry the action (GET/POST/PUT/PATCH/DELETE).
- Status codes are the contract — never 200-with-error-body.
- Paginate everything; prefer cursor pagination for live data.
- Version explicitly or evolve additively — never break a shipped field.
- HATEOAS links let clients discover next actions instead of hardcoding URLs.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev